

Plan-Then-Compile: Resolving Clinical Terminology in the Database Enables FHIR-to-SQL Generalization to Unseen Concepts

Anonymous Author(s)

Abstract

Text-to-SQL over clinical data conflates two skills: composing a query, and mapping a clinical concept to the code the database stores it under. When gold SQL embeds literal codes, the second is memorization, and no metric computed on that data can separate them. We rebuild a FHIR-derived benchmark so terminology is resolved *through the database* — every gold query looks the concept up in a `valuesets` table at runtime and never embeds a literal SNOMED CT, ICD-10 or RxNorm code — and pair each target with a structured JSON query plan compiled into SQL in one completion. On an earlier version of this pipeline that did embed literal codes, a fine-tuned model reached 98.4% execution match on familiar concepts but 20.7% on concepts absent from training. Under the lookup design, DoRA fine-tuning of a 14B open-weight coder model reaches 100% on familiar concepts and 89.1% on unseen ones (81.4% by the stricter text metric), from frozen baselines of 34.8% and 60.8%. An ablation isolates why this works: merely making the lookup table *visible* in the schema drives the *untrained* model’s code-hardcoding from 3.4% to 0.0%, with no training signal at all — models bypass a resolution mechanism they cannot see. We additionally report that a DAPO/GRPO stage adds nothing on top of supervised fine-tuning, which already saturates the reward’s correctness gate.

Keywords: text-to-SQL, clinical natural language processing, electronic health records, FHIR, large language models, parameter-efficient fine-tuning, clinical terminology, synthetic health data

Data and Code Availability This study uses only synthetic patient records generated by Synthea (Walonoski et al., 2018); no real patient data is involved. The corpus comprises two disjoint synthetic populations (18,999 training and 6,383 held-out patients) flattened into a relational schema. All

code, generated datasets, trained adapter weights, and the complete per-seed training logs, evaluation summaries, failure logs and ablation results behind every number reported here are publicly released under open licenses. Links are withheld here to preserve anonymity during review and will be provided on acceptance; an anonymized copy is available to reviewers on request through the programme chairs.

Institutional Review Board (IRB) This research does not require IRB approval. It uses only synthetic patient records generated by Synthea and involves no human subjects, no real patient data, and no identifiable information.

1. Introduction

“How many patients with essential hypertension were prescribed a statin last year?” looks like one translation step. It decomposes into at least four: isolating the clinical entities, identifying the query archetype they imply, composing SQL over a normalized schema, and resolving each concept to the code the data actually uses. The first three are reasoning that should transfer. The fourth, done from parametric memory, is recall — it degrades for rare concepts, varies across coding systems (SNOMED CT, ICD-10, RxNorm, LOINC, CDT and CVX all appear in our corpus), and cannot be audited.

Conventional supervision collapses all four into an opaque mapping to a query string, which is a measurement problem as much as a capability one: when a model is wrong there is no way to tell which skill failed, and when it is right there is no way to tell whether it reasoned or recognized.

This is not hypothetical. An earlier iteration of our own pipeline embedded literal codes in gold SQL. A fine-tuned model then reached 98.4% execution match on familiar clinical concepts and **20.7%** on concepts absent from training — not a reasoning fail-

ure, but a model unable to know codes it had never been shown. Rebuilding the corpus around database-resolved terminology is the response, and the results below are the comparison.

Clinical text-to-SQL has an established line of work (Wang et al., 2020b; Lee et al., 2022, 2024), as do intermediate representations between question and query (Guo et al., 2019; Wang et al., 2020a; Pourreza and Rafei, 2023) and cross-domain benchmarks (Yu et al., 2018; Li et al., 2023). Our contribution is not a new learning algorithm but a corpus and evaluation design that makes one specific capability — generalization to clinical concepts absent from training — separable and measurable.

Contributions. (i) A FHIR-native corpus in which terminology resolution is a queryable operation rather than recall, with an evaluation design that isolates generalization to unseen clinical concepts from generalization to unseen patients. (ii) A soundness-by-construction detector for lookup bypass, and the finding that schema *visibility* alone eliminates hard-coding in an untrained model. (iii) An explicit accounting of how much of each evaluation arm is genuinely novel, and a quantified bound on how discriminative execution match actually is. (iv) A scoped null result with mechanism: once supervised finetuning saturates the reward’s correctness gate, a DAPO/GRPO stage (Yu et al., 2025; Shao et al., 2024) has nothing left to optimize.

2. Method

2.1. Terminology resolved in the database

Every coded table carries a `code/system/display` triple, and a `valuesets` table indexes every (`table_name`, `code`, `code_system`, `display`) combination in the corpus. Gold SQL never contains a literal clinical code; it resolves the concept at query time, as in:

```
WITH resolved AS (
  SELECT code, code_system FROM valuesets
  WHERE table_name = 'procedure'
  AND display ILIKE '%Depression screening%'
)
SELECT DISTINCT patient_id
FROM procedure, resolved
WHERE procedure.code = resolved.code
  AND procedure.system = resolved.code_system
  AND status = 'completed'
```

This needs no retrieval infrastructure, no tool calls and no prompt-format change — it is ordinary SQL over columns that already exist. The model need only produce a plausible search phrase; the database resolves it deterministically. Concepts whose display text matches more than one row are excluded from gold generation rather than resolved by an arbitrary tiebreak.

2.2. Plan, then compile

Each target is a structured JSON query plan followed by the compiled SQL in one continuous completion. The plan’s `entities` array names the concepts, their roles (primary vs. companion), clinical domain, and whether each needs a lookup; `constraints` and `aggregation` encode the archetype, drawn from a closed vocabulary where every value maps to a clause shape some template already needs. Because the plan precedes the SQL autoregressively, the model commits to its reading of the question before composing the query, and that commitment is recoverable from the output.

2.3. Detecting lookup bypass

A model could still embed a literal code and produce a correct query, which an execution-only metric cannot see. We flag any comparison of `code/system` (and their FHIR-prefixed variants) against a quoted literal. This is sound by construction: legitimate lookup CTEs filter only on `table_name` and `display`, and join against `identifiers`, never literals, so a match can only mean the lookup was bypassed. Validated against all 9,680 gold-SQL-bearing training rows with zero false positives.

3. Setup

Two disjoint synthetic populations were generated with Synthea (Walonoski et al., 2018) (train 18,999 patients; held-out 6,383; zero `patient_id` overlap, verified by set intersection). Questions come from an archetype × concept factory: 82 archetypes (64 answerable, 18 unanswerable) across four difficulty tiers plus operational, regulatory and quality-KPI categories, instantiated against concepts spanning the coverage range. Every answerable gold query was executed and kept only if it ran without error and returned at least one row, then checked for repeated-execution stability. Assembled: 10,696

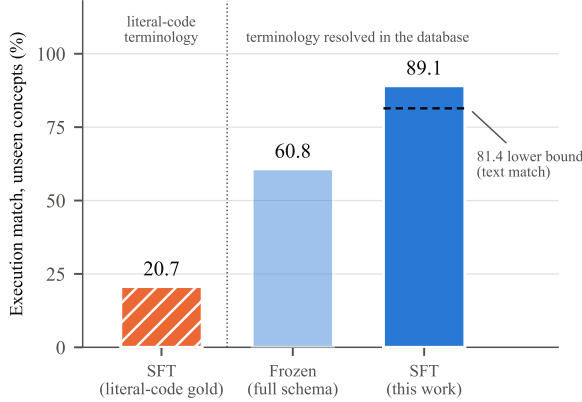


Figure 1: Execution match on clinical concepts absent from training. Left: an earlier iteration of this same pipeline, in which gold SQL embedded literal clinical codes. Right: the database-resolved design, frozen and fine-tuned, both given the complete schema. The dashed line marks the 81.4% lower bound implied by the stricter text-match metric (Section 5).

training rows (9,680 execution-verified + 1,016 unanswerable), 2,420 distinct gold queries.

We fine-tune **Qwen2.5-Coder-14B-Instruct** (Hui et al., 2024) with DoRA (Liu et al., 2024) (rank 16, α 32, all linear layers) under NF4 quantization (Dettmers et al., 2023), two epochs, three seeds, on a single consumer-cloud GPU (≈ 12 h/seed). A DAPO stage (Yu et al., 2025) then continues each seed’s best checkpoint. All queries execute against DuckDB (Raasveldt and Mühleisen, 2019).

Two evaluation arms. *Familiar* — concepts also used in training, against the disjoint population, isolating *population* generalization. *Unseen* — 84 concepts appearing nowhere in training (verified disjoint; zero shared question/query pairs), isolating *concept* generalization.

4. Results

Table 1 gives the headline, and Figure 1 isolates the effect of the terminology design on the arm that matters. A frozen model given the full schema still never reproduces a gold query (0.0% text match on both arms), refuses unanswerable questions unreliably in

Table 1: Held-out benchmark, mean over three seeds. Frozen figures use a *complete* schema description (Section 4.1); under the training-time prompt they are 10.2% and 31.3%.

	Frozen	SFT	SFT+RL
Exec. match, familiar	34.8	100.0	100.0
Exec. match, unseen	60.8	89.1	89.2
Text match, unseen	0.0	81.4	81.5
Hardcoding rate	0.0	0.0	0.0

both directions, and writes correct queries 4.5–12 $\times$ slower than gold. After fine-tuning, familiar-arm execution correctness is perfect on all three seeds; the unseen arm reaches 89.1% (85.0–92.1% across seeds).

4.1. Ablation: schema visibility alone removes hardcoding

During our published runs the schema shipped to the model omitted the **valuesets** DDL: fine-tuned models had learned the table’s shape from training targets, but the frozen baseline had only the table name. We re-ran the frozen model twice on identical rows, with and without that DDL.

Showing it the table quadruples familiar-arm execution match (8.7% $\rightarrow$ 34.8%) and roughly doubles unseen (31.2% $\rightarrow$ 60.8%). A fine-tuned control moves +0.8pp, confirming those models had already internalized the schema. Most striking: terminology-hardcoding falls to **0.0%** on both arms (from 1.8% and 4.2%). *An untrained model stops embedding literal codes purely because the lookup route became discoverable.* Models bypass a resolution mechanism they cannot see; the fix is as much schema design as training. The event counts are small (6 hardcoded completions across the without-DDL legs), so we report direction rather than effect size.

4.2. A scoped null: RL adds nothing once SFT saturates the objective

All three seeds hit their best dev execution match at the *first* evaluation checkpoint (step 10) and never improved, early-stopping at step 40 of a 200-step budget. Mean reward sat at or above 1.0 — the correct-answer floor — on 37–38 of 40 steps, meaning sampled completions were already overwhelmingly correct before any update. With correctness saturated, the reward variance Dynamic Sampling retained (it

still kept $\approx 87\%$ of groups) must have come largely from the small efficiency bonus, which is derived from wall-clock timing. We did not log per-group reward components, so this is an inference from reward statistics rather than a measurement.

5. What each arm can and cannot show

We report this explicitly because the headline invites over-reading. **96.5%** of the familiar arm’s answerable rows are byte-identical (question, gold SQL) pairs from training; only the patient population differs, and gold SQL holds no patient-specific literals. A purely memorizing model would also score 100% there. That arm shows learned queries *stay correct* on new data — necessary, not sufficient. The unseen arm shares zero question/query pairs and zero concepts with training, and is where transfer is established.

Its 89.1% is an **upper bound**. Unseen concepts are rare (1–3 patients each) and gold answers are small integers, so **53.9%** of within-archetype concept pairs return identical results — confusing one concept for another can go undetected. Text-match metrics cannot be satisfied that way and give 81.4% as a lower bound. True performance lies in 81–89%.

Finally, the 1,016 unanswerable questions are identical across training and both arms, so perfect abstention measures *retention* of trained refusal, not held-out refusal generalization.

6. Discussion

Why not simply call a frontier API? We deliberately do not benchmark against proprietary hosted models. For the deployment this targets — querying PHI-bearing clinical databases — transmitting records to a third-party API is frequently precluded by data-governance and residency rules regardless of accuracy, so a hosted baseline would not be an available alternative even if it matched. The relevant comparison is against what can run inside the institution’s own boundary. A locally hosted adapter also has bounded, one-off cost rather than per-query billing.

Limitations. Synthetic data only; all questions derive from one templated generation distribution, so robustness to free-form clinician phrasing is untested; terminology resolution is substring matching over a

dictionary with one display string per code, so synonymy, hierarchy and genuine ambiguity are out of scope (ambiguous concepts are excluded from gold) — real deployment needs value-set expansion; a single base model, scale and schema; and the unseen arm’s archetypes are all trained archetypes, so what transfers is concept identification and lookup composition, not novel query composition.

Outlook. Nothing here is specific to FHIR. Any domain with a normalized schema, a controlled vocabulary and a supply of verifiable queries presents the same structure. The most useful extension is not a larger model but a harder benchmark with genuine headroom above the supervised ceiling — which is where an RL stage might finally earn its cost.

References

- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.
- Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. Towards complex text-to-SQL in cross-domain database with intermediate representation. In *Proceedings of ACL*, 2019.
- Binyuan Hui, Jian Yang, Zeyu Cui, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Gyubok Lee, Hyeonji Hwang, Seongsu Bae, Yeonsu Kwon, Woncheol Shin, Seongjun Yang, Minjoon Seo, Jong-Yeup Kim, and Edward Choi. EHRSQL: A practical text-to-SQL benchmark for electronic health records. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks)*, 2022.
- Gyubok Lee, Sunjun Kweon, Seongsu Bae, and Edward Choi. TrustSQL: Benchmarking text-to-SQL reliability with penalty-based scoring. *arXiv preprint arXiv:2403.15879*, 2024.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, et al. Can LLM already serve as a database interface? a BIG bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks)*, 2023.

- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *Proceedings of ICML*, 2024.
- Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems*, 2023.
- Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, 2019.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeek-Math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Jason Walonoski, Mark Kramer, Joseph Nichols, Andre Quina, Chris Moesel, Dylan Hall, Carlton Duffett, Kudakwashe Dube, Thomas Gallagher, and Scott McLachlan. Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. *Journal of the American Medical Informatics Association*, 25(3):230–238, 2018.
- Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In *Proceedings of ACL*, 2020a.
- Ping Wang, Tian Shi, and Chandan K. Reddy. Text-to-SQL generation for question answering on electronic medical records. In *Proceedings of The Web Conference (WWW)*, 2020b.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of EMNLP*, 2018.